

Games Engineering Report

Jade(Yujie) Liang
5709365

University of Warwick

September 29, 2026

github:<https://github.com/Jadey0804/SunkenCity>

1 Game Design Document

1.1 Game Concept and Core Mechanics

SunkenCity is a third-person exploration and puzzle game developed using Unreal Engine 5. The player takes the role of a treasure hunter who travels to an island to search for a underwater treasure.

The core gameplay loop is:

Explore the environment, discover an obstacle or puzzle , use a tool or object , solve the puzzle , unlock the next area.

The player's main actions include movement, swimming, interaction, throwing, carrying heavy objects, activating pressure plates, and operating elevator mechanisms.

The focus of this game is on level design. The general game progression is as follows: the player arrives on the island, travels to the lighthouse, finds a map, and identifies the approximate location of the treasure. The player then solves a series of mechanism-based puzzles while searching for a route and eventually enters the underwater area to find the treasure.

The game is completed when the player finds the treasure underwater. The game is lost when the player runs out of oxygen. The game mainly maintains player engagement through environmental exploration, physical interaction, gradually increasing puzzle complexity, oxygen pressure, and visual feedback.

1.2 Narrative and Art Direction

The game has a relatively simple story. The player is a treasure-hunting enthusiast who hears that there is still undiscovered treasure near the island and therefore travels to the island alone to explore.

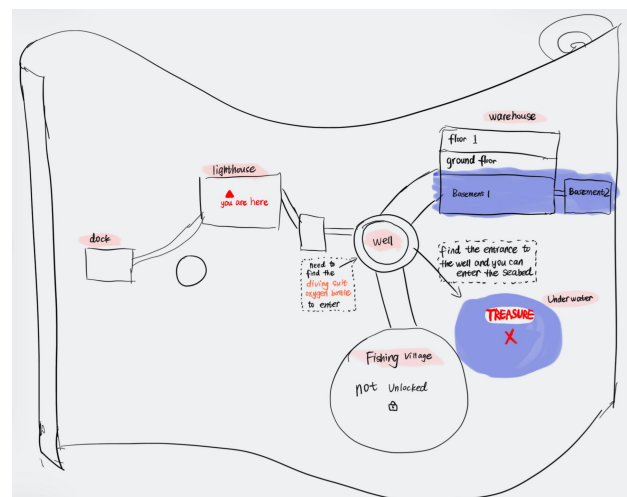


Figure 1: Game Map

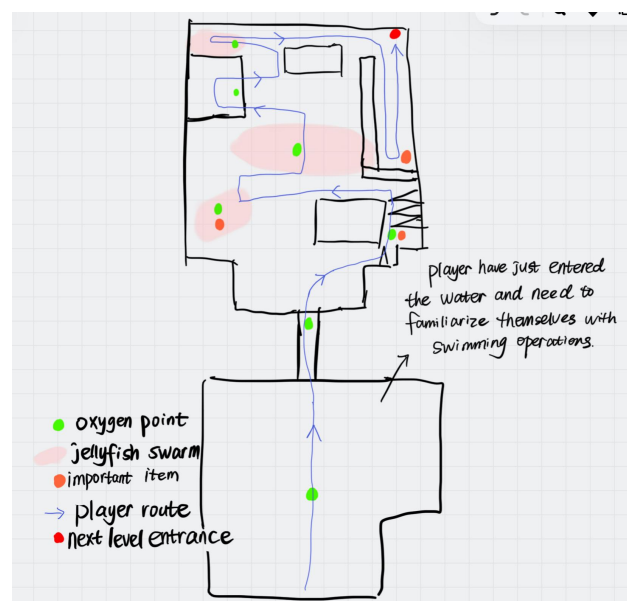


Figure 2: One of the level designs (warehouse basement)

The game uses a stylised visual style. This style is suitable for presenting the atmosphere of the island, lighthouse, underwater ruins, and treasure exploration. However, most of the game is currently still in the blackout stage and is being used as a placeholder to test visual guidance.

The lighting, water, ruins, fog, buildings, and environmental sound effects are mainly used to create a damp, dangerous, but explorable atmosphere. Underwater fish schools, oxygen bubbles, and ruin structures are also used to guide the player and indicate the importance of the area in which the player is currently located.

2 Technical Design Document

2.1 Technical Overview and Project Architecture

The game uses the following technologies and tools:

- **Game Engine:** Unreal Engine 5
- **Programming Methods:** C++ and Blueprints
- **Target Platform:** Windows PC
- **Other Tools:** Git, Rider, Fab, the UE5 Water plugin, and Marketplace assets

The project uses a structure that combines component-based design, object-oriented design, and event-driven design. The main functions are divided into player components, interactable Actors, puzzle Actors, UI Widgets, and AI Actors.

The game logic is mainly implemented through broadcast events, collision events, overlap events, input events, interfaces, and Blueprint events. Reusable functions are separated into independent components and interfaces, such as the interaction, throwing, oxygen, and item stimulus systems, allowing them to be reused across different Actors.

2.2 Key Game Systems

Interaction System The project uses an InteractionComponent to detect interactable objects near the player and calls the interaction logic of different Actors through the Interactable interface. This system is responsible for displaying interaction prompts and selecting the most suitable current interaction target.

Throwing System The ThrowableComponent implements item equipping, aiming, trajectory prediction, landing-point preview, and throwing. The player can throw small objects to activate distant mechanisms or affect the environment, such as scaring away or attracting animals. Interactable items are also divided into multiple classes and are used to affect different situations.

Carrying and Pressure Plate System The HeavyCarryableActor implements the picking up, carrying, and dropping of heavy objects. The PressurePlateActor detects whether a heavy object or throwable object has

entered the pressure area and then triggers a change in the state of the mechanism. This system is used to create puzzles involving placing heavy objects, holding down mechanisms, and unlocking routes.

Mechanism System The project implements pressure plates and a counterweight elevator. The CounterweightElevatorActor controls platform movement through a spline path and can pause or continue moving according to the state of an obstacle. It is used to connect player actions with level progression.

Oxygen System The OxygenComponent manages the player's underwater oxygen. Oxygen is continuously consumed while the player is diving, and sprinting increases the rate of oxygen consumption. The player can restore oxygen by entering an OxygenBubblePoint. Running out of oxygen causes the player to fail, so this system creates time pressure during underwater exploration.

Fish School AI System The FishSchoolActor uses Instanced Static Meshes to display a large number of fish and uses Boids behaviour to implement fish-school movement. The fish school includes separation, alignment, and cohesion logic. The fish scatter when the player approaches and regroup afterwards.

The external resources used in the project mainly include Fab assets, Marketplace art assets, and the UE5 Water plugin. These are used for the environment, water, characters, and scene construction.

3 Key Feature and Research Basis

The main feature from the critical review that was applied to this project is the Boids fish-school AI.

The core idea of Boids is to create complex group behaviour through simple individual rules. This project applies this technique to underwater fish schools. Each fish performs separation, alignment, and cohesion based on the positions and velocities of nearby fish, while also producing fleeing behaviour based on the player's position.

This feature mainly improves the following aspects:

- **Gameplay Depth:** The fish schools are not a core gameplay mechanic, but they improve spatial feedback in the underwater area by allowing the player to see the environment react to their movement.

- **Player Engagement:** The fish scatter when the player approaches, making underwater exploration feel more alive and reducing the sense of emptiness.

- **Visual and Technical Quality:** Boids improve the dynamic presentation of the underwater environment.

For example, in the underwater treasure-hunting area, the fish scatter when the player swims through the school and gradually regroup afterwards. This effect does not directly determine the solution to a puzzle, but it strengthens the feeling that the player has entered a realistic underwater ecosystem.

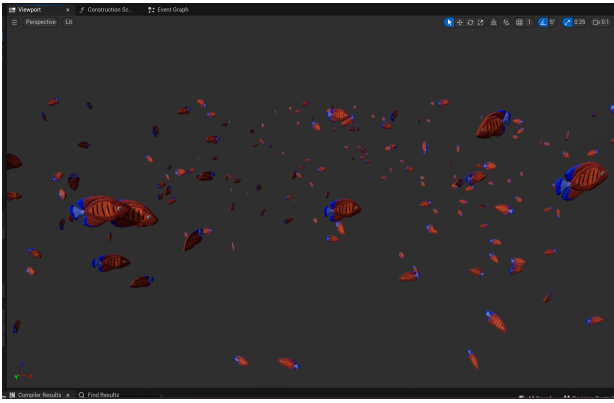


Figure 3: *FishSchoolActor Viewport*

The game is also planned to be improved further in the future by adding shark AI to the ocean. The sharks will chase the player, and the Boids fish schools will also scatter when they encounter a shark. This can act as a visual warning. Overall, the AI fish-school system has strong potential for further expansion.

4 Testing, Debugging, and Iteration

Testing was carried out throughout the development process. The main methods included regular playtesting, individual mechanic testing, Blueprint breakpoints, Print String, collision visualisation, trajectory-preview inspection, and AI behaviour observation.

The main areas tested included:

- Whether the player could clearly see the objective and understand the interaction prompts.
- Whether throwing, carrying, and pressure plates could be used reliably to complete puzzles.
- Whether collision, overlap, and physics states worked correctly.
- Whether the oxygen UI clearly communicated dangerous conditions.
- Whether the game could progress from landing on the island to completing the underwater treasure-hunting sequence.

5 Project Evaluation and Future Development

The final project successfully demonstrates the core progression of a UE5 third-person exploration and puzzle game. The project implements throwing, carrying heavy objects, pressure plates, elevator mechanisms, an oxygen system, and Boids fish-school AI.

The main strengths of the project include:

- The core gameplay is clear and focuses on object interaction and environmental puzzles.

- The system structure is relatively clear, with interaction, throwing, oxygen, and fish-school functions remaining relatively independent.

- The underwater area uses oxygen and fish schools to improve atmosphere and feedback.

The current main limitations include:

- The number and variety of puzzles are still limited.
- The fish-school AI is mainly used for visual atmosphere and does not yet participate deeply in gameplay.
- Level guidance, animation presentation, and UI polish still have room for improvement.
- There are very few art assets and sound effects.

With more development time, the project would reorganise the currently implemented puzzle solutions into a set of puzzle mechanics that complement one another, conduct more player testing, improve character animations and underwater presentation, allow the Boids fish schools to participate further in route guidance or environmental puzzles, and add more complete save and accessibility settings.